

**Universidad Nacional de Río Cuarto**  
Facultad de Ciencias Exactas, Físico-Químicas y Naturales  
Departamento de Computación

## **Bases de Datos II**

### **Examen Final — 2017**

Resolución de estudio

**Alumno:** Tomás Rodeghiero  
**Año de estudio:** 2026

## Índice

|                                                                |    |
|----------------------------------------------------------------|----|
| 1. Introducción                                                | 2  |
| 2. Pregunta 1: Seguridad en SQL                                | 2  |
| 3. Pregunta 2: Protocolo de ordenamiento por marcas temporales | 4  |
| 4. Pregunta 3: Pasos en el procesamiento de una consulta SQL   | 6  |
| 5. Pregunta 4: Heurísticas en optimización de consultas        | 7  |
| 6. Pregunta 5: Algoritmo de Hash Join                          | 9  |
| Cierre                                                         | 11 |

## 1. Introducción

El examen 2017 consta de cinco preguntas teóricas. Tres ya aparecieron en exámenes anteriores (marcas temporales, procesamiento de consultas), pero se desarrollan aquí en formato completo para que la guía sea autocontenida. Las dos preguntas nuevas respecto del 2016 son seguridad en SQL (Pregunta 1) y heurísticas de optimización (Pregunta 4); la Pregunta 5 cambia de Merge Join (2016) a Hash Join (2017).

Mapa de las preguntas con los teóricos del cuatrimestre:

- Pregunta 1 → Teórico 2 + Práctico 2: seguridad/DCL.
- Pregunta 2 → Teórico 8 + Práctico 6 Ej. 3: protocolo de marcas temporales.
- Pregunta 3 → Teórico 6: pipeline de procesamiento.
- Pregunta 4 → Teórico 7: optimización heurística vs por costos.
- Pregunta 5 → Teórico 6: algoritmo de Hash Join.

## 2. Pregunta 1: Seguridad en SQL

### Consigna

Desarrolle seguridad en SQL.

### Idea

La seguridad en SQL es el subsistema que decide *quién puede hacer qué* contra la base de datos. Se compone de tres capas: **autenticación** (verificar la identidad del usuario que se conecta), **autorización** (qué privilegios tiene esa identidad sobre cada objeto) y **auditoría** (registro de quién hizo qué y cuándo). El subconjunto del lenguaje SQL que maneja la autorización se llama **DCL** (*Data Control Language*) y se centra en dos verbos: **GRANT** y **REVOKE**.

### Respuesta

#### 1. Identidades.

- **Usuarios:** cuentas individuales con credenciales (`CREATE USER nombre IDENTIFIED BY 'clave'`). En MySQL la identidad efectiva es el par `'usuario'@'host': 'juan'@'localhost'` y `'juan'@' %'` son cuentas distintas.
- **Roles** (PostgreSQL, MySQL 8+, Oracle): agrupaciones nombradas de privilegios. Se crean con `CREATE ROLE` y se asignan a usuarios con `GRANT rol TO usuario`. En PostgreSQL no hay diferencia entre usuario y rol; solo el flag `LOGIN` indica si una identidad puede iniciar sesión.

#### 2. Privilegios. El estándar define privilegios sobre objetos del esquema:

- **SELECT, INSERT, UPDATE, DELETE:** las cuatro operaciones DML básicas.
- **REFERENCES:** permite que la tabla del usuario apunte con una FK a esta tabla.
- **EXECUTE:** invocar funciones y procedimientos.
- **ALL PRIVILEGES:** atajo para todos los anteriores.

Los privilegios pueden ser de **grano fino**: `GRANT SELECT (apellido, nombre) ON cliente TO ...` otorga lectura solo de esas columnas, no de la tabla entera. `GRANT UPDATE (descripcion)` hace lo mismo para escritura. Oracle no soporta `SELECT(col)`; el workaround idiomático es crear una *vista* que proyecte solo las columnas autorizadas y otorgar `SELECT` sobre la vista.

### 3. GRANT y REVOKE.

```
-- Otorgar
GRANT SELECT, INSERT ON practico1a.cliente
  TO 'vendedor'@'localhost';

-- Otorgar con delegacion (el receptor puede a su vez otorgar)
GRANT UPDATE (descripcion) ON practico1a.producto
  TO 'admin'@'localhost'
  WITH GRANT OPTION;

-- Revocar
REVOKE INSERT ON practico1a.cliente
  FROM 'vendedor'@'localhost';

-- Revocar todo (limpieza total)
REVOKE ALL PRIVILEGES, GRANT OPTION
  FROM 'vendedor'@'localhost';
```

WITH GRANT OPTION permite cadenas de delegación: A le da el privilegio a B con opción de delegar; B se lo da a C. Si A luego revoca de B, en motores con CASCADE se revoca también de C.

4. *Vistas como mecanismo de seguridad.* Una *vista* proyectiva permite exponer un subconjunto controlado de columnas o filas:

```
CREATE VIEW vw_cliente_publico AS
SELECT nro_cliente, apellido, nombre FROM cliente;

GRANT SELECT ON vw_cliente_publico TO 'consultor'@'localhost';
-- El consultor no ve direccion ni telefono
```

### 5. Restricciones de conexión y recursos.

- En MySQL, ALTER USER ... WITH MAX\_CONNECTIONS\_PER\_HOUR n MAX\_QUERIES\_PER\_HOUR m.
- En Oracle, CREATE PROFILE ... LIMIT CONNECT\_TIME n y ALTER USER ... PROFILE p; ALTER USER ... PASSWORD EXPIRE fuerza renovación de contraseña.
- En PostgreSQL, el control de conexiones por host vive en `pg_hba.conf`; los recursos se limitan con extensiones (`pg_stat_statements`, `pgbouncer`).

6. *Inyección SQL y PreparedStatement.* Concatenar valores de usuario en el SQL es la vulnerabilidad más común: si el cliente envía `nombre = "Pedro"; DROP TABLE cliente; -"`, una concatenación ingenua ejecuta el DROP. La defensa es usar `PreparedStatement` con placeholders ? (en JDBC) o \$1, \$2 (en libpq): la sentencia y los valores viajan por separado, el motor nunca interpreta los valores como código SQL.

7. *Principio de mínimo privilegio.* Cada usuario debe tener solo los permisos estrictamente necesarios. El idioma típico es: `vendedor` solo INSERT sobre `factura`; `administrador` solo UPDATE(`descripcion`) sobre `producto`; no se otorga ALL PRIVILEGES salvo a cuentas administrativas.

8. *Verificación.* Los catálogos exponen los permisos vigentes:

- MySQL: SHOW GRANTS FOR 'usuario'@'host';
- PostgreSQL: `information_schema.role_table_grants`, `column_privileges`, `pg_roles`, funciones `has_table_privilege`, `has_column_privilege`.
- Oracle: `DBA_TAB_PRIVS`, `DBA_ROLE_PRIVS`, `DBA_USERS`.

## Ejemplo

Escenario típico de práctico: tres usuarios sobre la base `practico1a_mysql` con privilegios distintos.

```
-- Crear usuarios
CREATE USER 'vendedor1'@'localhost' IDENTIFIED BY 'Vendedor1!';
CREATE USER 'vendedor2'@'localhost' IDENTIFIED BY 'Vendedor2!';
CREATE USER 'administrador'@'localhost' IDENTIFIED BY 'Admin!';

-- vendedor1: solo puede insertar clientes (sin delegacion)
GRANT INSERT ON practico1a_mysql.cliente
TO 'vendedor1'@'localhost';

-- vendedor2: insertar facturas y leer columnas de producto
GRANT INSERT ON practico1a_mysql.factura
TO 'vendedor2'@'localhost';
GRANT SELECT (cod_producto, descripcion, precio)
ON practico1a_mysql.producto
TO 'vendedor2'@'localhost';

-- administrador: actualizar la descripcion del producto, delegable
GRANT UPDATE (descripcion) ON practico1a_mysql.producto
TO 'administrador'@'localhost'
WITH GRANT OPTION;

-- Todos pueden borrar clientes
GRANT DELETE ON practico1a_mysql.cliente
TO 'vendedor1'@'localhost',
'vendedor2'@'localhost',
'administrador'@'localhost';

-- Verificacion
SHOW GRANTS FOR 'administrador'@'localhost';

-- Revocar todo a vendedor2 al final del ciclo
REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'vendedor2'@'localhost';
```

## 3. Pregunta 2: Protocolo de ordenamiento por marcas temporales

### Consigna

Explique el protocolo de ordenamientos por Marcas temporales. Ejemplifique.

### Idea

El protocolo de *marcas temporales* (*timestamp ordering*) es un mecanismo **pesimista**, **no basado en bloqueos** para garantizar *serializabilidad por conflicto*. Cada transacción  $T_i$  recibe al ingresar al sistema una marca temporal  $MT(T_i)$  única y monótonamente creciente. El protocolo garantiza que la planificación resultante sea equivalente al orden serial de las marcas: si  $MT(T_i) < MT(T_j)$ , todo conflicto se resuelve como si  $T_i$  se hubiera ejecutado antes que  $T_j$ . A diferencia de 2PL, no produce deadlocks (no hay esperas), pero sí puede generar rollbacks en cascada y starvation.

### Respuesta

*Estructuras que mantiene el protocolo.* Cada ítem  $Q$  tiene dos marcas:

- $MT-E(Q)$ : mayor marca temporal de las transacciones que escribieron  $Q$  con éxito.
- $MT-L(Q)$ : mayor marca temporal de las transacciones que leyeron  $Q$  con éxito.

*Reglas de validación.*

$T_i$  ejecuta  $r(Q)$ :

- Si  $MT(T_i) < MT-E(Q) \Rightarrow$  rechazo.  $T_i$  querría leer una versión ya sobrescrita; rollback y reinicio con marca nueva.
- En otro caso  $\Rightarrow$  se ejecuta y  $MT-L(Q) \leftarrow \max(MT-L(Q), MT(T_i))$ .

$T_i$  ejecuta  $w(Q)$ :

- Si  $MT(T_i) < MT-L(Q) \Rightarrow$  rechazo. El valor de  $T_i$  ya fue necesario por una lectura posterior; rollback.
- Si  $MT(T_i) < MT-E(Q) \Rightarrow$  rechazo.  $T_i$  querría escribir una versión obsoleta; rollback.
- En otro caso  $\Rightarrow$  se ejecuta y  $MT-E(Q) \leftarrow MT(T_i)$ .

*Propiedades.*

- Garantiza serializabilidad por conflicto; el orden serial equivalente es el de las marcas.
- Libre de deadlocks: no hay esperas.
- Sufre rollbacks en cascada si combinamos con escrituras inmediatas; se mitiga con escritura diferida hasta commit.
- Posible starvation: una transacción larga puede ser abortada repetidamente si llega tarde respecto del orden global.

## Ejemplo

Sean  $T_1, T_2, T_3$  con  $MT = 1, 2, 3$ .

- $T_1$ :  $r(X)$ ;  $w(X)$ ;  $r(Y)$ ;  $w(Y)$ ;
- $T_2$ :  $r(X)$ ;  $w(X)$ ;
- $T_3$ :  $r(Y)$ ;  $w(Y)$ ;

Estado inicial:  $MT-E = MT-L = 0$  para  $X$  e  $Y$ .

Schedule:  $r_1(X), w_1(X), r_2(X), w_2(X), r_1(Y), w_1(Y), r_3(Y), w_3(Y)$ .

| Paso | Op.      | Validación    | $MT-E(X)$ | $MT-L(X)$ | $MT-E(Y)$ | $MT-L(Y)$ |
|------|----------|---------------|-----------|-----------|-----------|-----------|
| 0    | inicio   | —             | 0         | 0         | 0         | 0         |
| 1    | $r_1(X)$ | $1 \geq 0$ OK | 0         | 1         | 0         | 0         |
| 2    | $w_1(X)$ | OK            | 1         | 1         | 0         | 0         |
| 3    | $r_2(X)$ | $2 \geq 1$ OK | 1         | 2         | 0         | 0         |
| 4    | $w_2(X)$ | OK            | 2         | 2         | 0         | 0         |
| 5    | $r_1(Y)$ | $1 \geq 0$ OK | 2         | 2         | 0         | 1         |
| 6    | $w_1(Y)$ | OK            | 2         | 2         | 1         | 1         |
| 7    | $r_3(Y)$ | $3 \geq 1$ OK | 2         | 2         | 1         | 3         |
| 8    | $w_3(Y)$ | OK            | 2         | 2         | 3         | 3         |

Sin rollbacks; orden serial equivalente  $T_1, T_2, T_3$ .

*Caso con rollback.* Si el schedule fuera  $r_2(X), w_2(X), w_1(X)$ : en el paso 3,  $MT(T_1) = 1 < MT-E(X) = 2 \Rightarrow$  rechazo,  $T_1$  aborta y reinicia con marca nueva ( $MT(T_1) = 4$  por ejemplo).

## 4. Pregunta 3: Pasos en el procesamiento de una consulta SQL

### Consigna

Explique los diferentes pasos en el procesamiento de una consulta SQL por parte de un motor de bases de datos. Dé un ejemplo de una consulta como se va transformando en el proceso.

### Idea

Una consulta SQL no se ejecuta tal como llega del cliente: el motor la atraviesa por un pipeline de cinco etapas que convierten progresivamente texto  $\rightarrow$  árbol sintáctico  $\rightarrow$  árbol algebraico  $\rightarrow$  plan físico  $\rightarrow$  ejecución. Las primeras etapas son determinísticas (parsing, validación); las del medio (optimización lógica y física) son donde el motor elige entre planes equivalentes basándose en estadísticas; la última (ejecución) llama al storage engine.

### Respuesta

*Pipeline canónico:*

1. **Parsing.** El parser tokeniza el texto y arma un *árbol sintáctico*. Detecta errores de sintaxis.
2. **Validación semántica.** Consulta el catálogo (`pg_catalog`, `INFORMATION_SCHEMA`) para verificar que las tablas y columnas existen, que los tipos son compatibles y que el usuario tiene permisos. Resuelve identificadores ambiguos.
3. **Traducción a álgebra relacional.** El árbol sintáctico se mapea a operadores algebraicos: `FROM`  $\rightarrow$  producto/join, `WHERE`  $\rightarrow \sigma$ , `SELECT`  $\rightarrow \Pi$ , `GROUP BY`  $\rightarrow \gamma$ , `ORDER BY`  $\rightarrow$  sort. Resultado: *árbol de consulta inicial*.
4. **Optimización.**
  - *Lógica (heurística):* aplica reglas de equivalencia (push-down de selecciones y proyecciones, reordenamiento de joins). Produce el *árbol algebraico optimizado*.
  - *Física (por costos):* para cada operador elige un algoritmo concreto (índice vs scan, nested loop vs hash vs merge join) usando estadísticas. Produce el *plan de ejecución*.
5. **Ejecución.** El plan se traduce a iteradores; cada uno pide tuplas al de abajo en pipeline; el storage engine sirve bloques.

Las etapas 1–2 fallan con errores SQL clásicos (`syntax error`, `column does not exist`). La etapa 4 es la que muestra `EXPLAIN/EXPLAIN ANALYZE`. Para influir en la etapa 4 hay tres palancas: agregar/quitar índices, refrescar estadísticas con `ANALYZE`, o reescribir la consulta.

### Ejemplo

Consulta:

```
SELECT c.nombre, c.apellido, f.monto
FROM cliente c
JOIN factura f ON c.nro_cliente = f.nro_cliente
WHERE f.monto > 100000
ORDER BY f.monto DESC;
```

*Paso 1 — Parsing.* Árbol sintáctico con nodos *Project / Join / Filter / Sort*.

*Paso 2 — Validación.* Catálogo: `cliente.nombre`, `cliente.apellido`, `factura.monto`, `factura.nro_cliente` existen y son compatibles.

*Paso 3 — Álgebra (inicial):*

$$\Pi_{\text{nombre, apellido, monto}}(\sigma_{f.monto > 100000}(\text{cliente} \bowtie_{\text{nro\_cliente}} \text{factura}))$$

Paso 4a — Optimización lógica. Push-down del WHERE a la rama de factura:

$$\Pi_{\dots}(cliente \bowtie \sigma_{monto > 100000}(factura))$$

Paso 4b — Optimización física. Con índice sobre factura.monto y PK en cliente.nro\_cliente:

```
Sort (key=f.monto DESC)
-> Nested Loop
    -> Index Scan using idx_factura_monto on factura f
        Index Cond: (f.monto > 100000)
    -> Index Scan using pk_cliente on cliente c
        Index Cond: (c.nro_cliente = f.nro_cliente)
```

Paso 5 — Ejecución. El Index Scan filtra factura; por cada match, el Nested Loop hace un lookup en cliente; el Sort acumula y emite ordenado.

## 5. Pregunta 4: Heurísticas en optimización de consultas

### Consigna

¿Qué es una heurística? ¿Cuáles se utilizan en optimización de consultas?

### Idea

Una **heurística** es una regla práctica que, sin garantía formal, *mejora el resultado en la mayoría de los casos*. En optimización de consultas las heurísticas se usan en la etapa *lógica* del optimizador (Teórico 7): reescriben el árbol algebraico aplicando reglas de equivalencia que casi siempre bajan el costo, sin necesidad de mirar estadísticas. Son complementarias a la optimización *por costos*, que sí consulta el catálogo para elegir el plan físico más barato entre los candidatos.

### Respuesta

*Heurística vs optimización por costos.*

| Aspecto   | Heurística                     | Por costos                             |
|-----------|--------------------------------|----------------------------------------|
| Entrada   | Árbol algebraico               | Árbol + estadísticas catálogo          |
| Salida    | Árbol equivalente mejor        | Plan físico de menor costo             |
| Garantía  | “Casi siempre mejora”          | Óptimo en el espacio considerado       |
| Velocidad | Muy rápida (reglas locales)    | Lenta (enumera planes)                 |
| Ejemplo   | “Empujar $\sigma$ a las hojas” | “Usar índice si selectividad $< 5\%$ ” |

En la práctica los motores los combinan: primero heurística para podar el espacio de búsqueda, después costos para elegir el mejor plan físico.

*Reglas heurísticas canónicas (Teórico 7).*

1. **Empujar selecciones hacia las hojas.** Si una selección  $\sigma_{\theta}$  involucra atributos de una sola relación, se baja para que se evalúe antes de los joins. Achica la cardinalidad de los intermedios.

$$\sigma_{\theta}(E_1 \bowtie E_2) \equiv \sigma_{\theta}(E_1) \bowtie E_2 \quad \text{si } \theta \text{ refiere solo a } E_1$$

2. **Cascada de selecciones.** Una  $\sigma$  conjuntiva se rompe en cadena para que cada predicado se pueda mover de manera independiente:

$$\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$



3. **Conmutatividad de selecciones.** Dos  $\sigma$  consecutivas pueden permutarse sin cambiar el resultado:

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

Útil para ejecutar primero la más selectiva (la que descarta más filas).

4. **Push-down de proyecciones.** Una  $\Pi$  se baja hasta que solo queden los atributos necesarios para el resto del árbol. Achica el ancho de tupla y, por lo tanto, el costo de I/O.

$$\Pi_L(E_1 \bowtie E_2) \equiv \Pi_L(\Pi_{L_1}(E_1) \bowtie \Pi_{L_2}(E_2))$$

5. **Reordenamiento de joins.** Joins son conmutativos ( $E_1 \bowtie E_2 \equiv E_2 \bowtie E_1$ ) y asociativos ( $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$ ). La heurística pide ejecutar primero los joins cuyo resultado intermedio sea más pequeño (típicamente cuando uno de los atributos es clave en una de las relaciones).
6. **Reemplazar producto cartesiano + selección por join.** Si tras los pasos anteriores quedó  $\sigma_{\theta}(E_1 \times E_2)$  con  $\theta$  vinculando ambos, se transforma en  $E_1 \bowtie_{\theta} E_2$ , que ejecuta con algoritmo de join (mucho más barato).
7. **Eliminación de subexpresiones comunes.** Si un subárbol aparece dos veces en la consulta (ej. una vista referenciada dos veces), se computa una sola vez y se reutiliza.
8. **Materialización temprana de subconsultas.** Subconsultas correlacionadas se transforman en joins cuando es posible (*decorrelation*).

*Por qué funcionan estas reglas.* Las primeras cinco son las que dominan en la práctica: *cualquier* reducción de cardinalidad o ancho de tupla antes de un join se amplifica en las etapas siguientes. Una selección que descarta el 90 % de las filas de la rama **factura** achica el join por 10x; si está en un nodo bajo, el efecto se propaga al resto del plan.

## Ejemplo

Consulta sobre el esquema del Práctico 1.a:

```
SELECT c.apellido, f.nro_factura
FROM cliente c, factura f
WHERE c.nro_cliente = f.nro_cliente
      AND f.monto > 100000
      AND c.direccion LIKE 'San Martin%';
```

Árbol inicial (sin optimizar):

$$\Pi_{\text{apellido, nro\_factura}}(\sigma_{\theta}(\text{cliente} \times \text{factura}))$$

donde  $\theta = \text{c.nro\_cliente} = \text{f.nro\_cliente} \text{ AND } \text{f.monto} > 100000 \text{ AND } \text{c.direccion LIKE 'San Martin\%'}$ .

Paso 1 — Cascada (R1):  $\sigma$  se rompe en tres:

$$\sigma_{\text{c.nro\_cliente=f.nro\_cliente}}(\sigma_{\text{f.monto>100000}}(\sigma_{\text{c.direccion LIKE 'San Martin\%'}}(\text{cliente} \times \text{factura})))$$

Paso 2 — Push-down (regla 1): cada  $\sigma$  baja a la rama donde están sus atributos:

$$\sigma_{\text{c.nro\_cliente=f.nro\_cliente}}(\sigma_{\text{c.direccion LIKE \dots}}(\text{cliente}) \times \sigma_{\text{f.monto>100000}}(\text{factura}))$$

*Paso 3 — Producto+selección  $\rightarrow$  join (regla 6):*

$$\sigma_{c.direccion \text{ LIKE } \dots (cliente)} \bowtie_{nro\_cliente} \sigma_{f.monto > 100000}(factura)$$

*Paso 4 — Push-down de proyección (regla 4):*

$$\Pi_{apellido, nro\_factura}(\Pi_{nro\_cliente, apellido}(\sigma_{\dots}(cliente)) \bowtie \Pi_{nro\_cliente, nro\_factura}(\sigma_{\dots}(factura)))$$

Cada operador trabaja con tuplas más angostas; el join se hace sobre filas filtradas y con menos columnas. Sin estas heurísticas, el motor habría calculado primero  $cliente \times factura$  (potencialmente millones de tuplas) y filtrado después, multiplicando el costo varias veces.

## 6. Pregunta 5: Algoritmo de Hash Join

### Consigna

Explique el algoritmo de hash join.

### Idea

*Hash Join* es uno de los tres algoritmos físicos canónicos para resolver un equi-join  $r \bowtie_A s$  (junto con Nested Loop y Merge Join). Su idea es ahorrar la comparación cuadrática: en vez de comparar cada tupla de  $r$  contra cada tupla de  $s$ , se construye una **tabla hash** sobre la relación más chica y se la consulta tupla por tupla con las de la relación más grande. Solo funciona para *equi-joins* (igualdad sobre el atributo de join); no soporta condiciones de rango ni  $\neq$ .

### Respuesta

*Variantes.*

1. **In-memory hash join (Classic Hash Join).** La relación más chica (*build*) cabe en memoria. Caso ideal.
2. **Grace Hash Join.** La build no cabe en memoria: se particionan *ambas* relaciones en archivos en disco usando la misma función hash, y luego se hace classic hash join por cada partición.
3. **Hybrid Hash Join.** Mejora sobre Grace: mantiene la primera partición en memoria mientras se escribe el resto a disco, evitando una vuelta de I/O.

*Algoritmo (classic, build + probe):*

*Build phase.* Por cada tupla  $t \in r$  (build, la chica), calcular  $h(t.A)$  e insertarla en la celda correspondiente de la tabla hash  $H$ .

*Probe phase.* Por cada tupla  $u \in s$  (probe, la grande), calcular  $h(u.A)$ , recorrer las tuplas de  $H[h(u.A)]$  y, por cada  $t$  en esa celda con  $t.A = u.A$ , emitir el par  $(t, u)$ .

Pseudocódigo:

```
// Build
H = new HashTable
for t in r:
    H.insert(h(t.A), t)

// Probe
```

```

for u in s:
    bucket = H.lookup(h(u.A))
    for t in bucket:
        if t.A == u.A:                // chequeo final por colisiones
            output (t, u)

```

*Análisis de costo.*

- **Classic hash join** (build cabe en memoria):

$$\text{Costo} \approx b_r + b_s \quad (\text{transferencias})$$

Una pasada por cada relación.

- **Grace hash join** con  $M$  buffers en memoria y  $b_r \leq M^2$ :

$$\text{Costo} \approx 3(b_r + b_s)$$

Cada relación se lee, se escribe particionada y se vuelve a leer en la fase de match.

- **Hybrid hash join**: entre  $b_r + b_s$  y  $3(b_r + b_s)$ , dependiendo de cuánto cabe de la primera partición en memoria.

*Propiedades.*

- **Asimétrico**: la relación elegida como build debe ser la más chica, porque la tabla hash vive en memoria. Si se equivoca el optimizador, la build no cabe y hay que ir a Grace.
- **Solo equi-join**: la igualdad del atributo de join es esencial para que el hash agrupe los matches; rangos no se pueden hashear.
- **No preserva orden** de salida (a diferencia de Merge Join). Si después hay ORDER BY, el motor agrega un *Sort* arriba.
- **Excelente para discos modernos** y memorias grandes: la build cabe casi siempre, y la probe es un scan secuencial sobre la grande.
- **Sensible a la distribución de los datos**: si muchas tuplas de  $s$  tienen el mismo valor de  $A$ , las celdas correspondientes se vuelven largas y la probe degrada a búsqueda lineal dentro del bucket.

*Decisión del optimizador (cuándo se elige Hash Join).*

- Cuando una relación es claramente más chica y cabe (parcialmente) en memoria.
- Cuando no hay índices ordenados utilizables para Merge Join.
- Cuando los inputs son grandes y un Nested Loop sería cuadrático.
- Cuando el atributo de join es una igualdad pura.

## Ejemplo

Sean dos relaciones:

| $r.A$ | $r.B$ | $s.A$ | $s.C$ |
|-------|-------|-------|-------|
| 1     | a     | 1     | x     |
| 2     | b     | 1     | y     |
| 3     | c     | 2     | z     |
|       |       | 4     | w     |

$r$  es más chica  $\Rightarrow$  build sobre  $r$ , probe con  $s$ .

*Build phase.* Asumiendo  $h(v) = v \bmod 4$ :

| Celda $h$ | Contenido |
|-----------|-----------|
| 0         |           |
| 1         | (1, a)    |
| 2         | (2, b)    |
| 3         | (3, c)    |

*Probe phase.* Por cada  $u \in s$ :

- $u = (1, x)$ :  $h(1) = 1 \rightarrow$  celda contiene (1, a). Igualdad real ( $1 = 1$ )  $\Rightarrow$  emite (1, a, x).
- $u = (1, y)$ :  $h(1) = 1 \rightarrow$  misma celda. Emite (1, a, y).
- $u = (2, z)$ :  $h(2) = 2 \rightarrow$  celda contiene (2, b). Emite (2, b, z).
- $u = (4, w)$ :  $h(4) = 0 \rightarrow$  celda vacía  $\Rightarrow$  no emite nada.

Resultado:  $\{(1, a, x), (1, a, y), (2, b, z)\}$ . Total: 3 lecturas de  $r$  (build) + 4 lecturas de  $s$  (probe) =  $b_r + b_s$  transferencias.

**En MySQL 8+.** Hash Join llegó en MySQL 8.0.18 (octubre 2019) y, desde 8.0.20, se aplica también a equi-joins con índices cuando el optimizador estima que es más barato que Merge Join o Nested Loop. En MySQL 5.7 solo existían Nested Loop y Block Nested Loop. Para forzar el algoritmo: hints `/*+ HASH_JOIN(t1, t2) */` y `/*+ NO_HASH_JOIN(t1, t2) */`.

## Cierre

El examen 2017 cubre dos focos: seguridad y procesamiento/optimización.

- **Seguridad** (Pregunta 1): la base es DCL (`GRANT/REVOKE`) con privilegios de grano fino, roles y vistas. El principio de mínimo privilegio y el uso de `PreparedStatement` contra SQL Injection son los dos pilares prácticos.
- **Concurrencia** (Pregunta 2): marcas temporales como alternativa pesimista a 2PL.
- **Procesamiento + optimización** (Preguntas 3, 4, 5): pipeline del motor, heurísticas de optimización lógica (push-down de  $\sigma$  y  $\Pi$ , reordenamiento de joins) y algoritmos físicos (Hash Join).

La idea común detrás del examen es que *el motor no es una caja negra*: tanto las decisiones de seguridad (qué le doy a quién) como las de performance (qué plan elige el optimizador) están explicitadas en SQL y se pueden inspeccionar con `SHOW GRANTS`, `EXPLAIN ANALYZE` y el catálogo. Saber cómo funcionan por dentro es lo que separa al “usuario de SQL” del “programador de bases de datos”.